

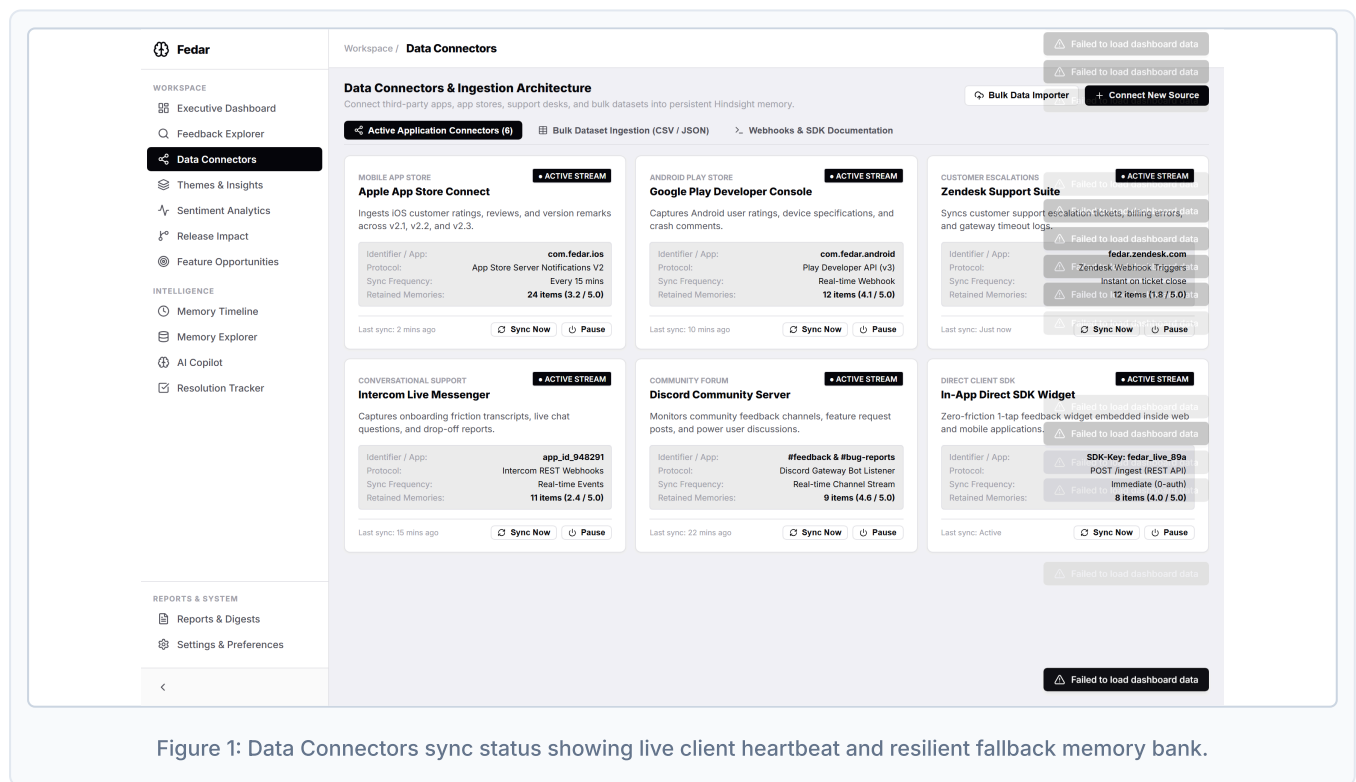
Building a Resilient Local Memory Engine with Hindsight and Sentence Transformers

When building stateful AI agents for enterprise environments, cloud dependencies can be a double-edged sword. If your memory bank requires a 500ms network roundtrip to a remote vector database on every single recall, interactive user experiences grind to a halt. Even worse, if your remote memory cluster experiences network downtime or rate limiting, your entire ingestion pipeline crashes.

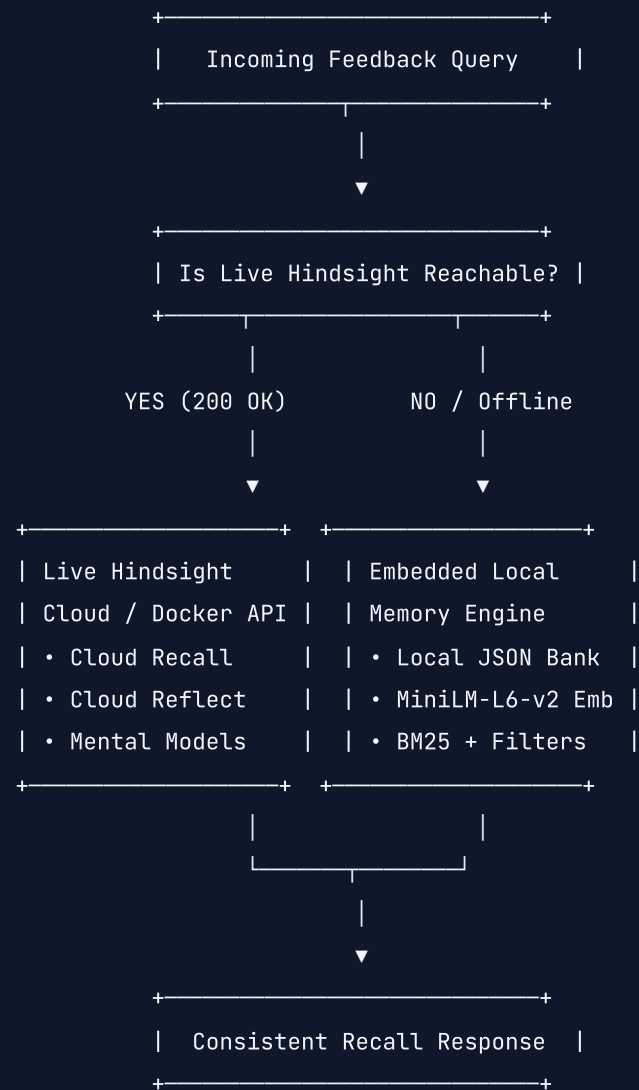
When architecting **Fedar**, we designed our core client— `HindsightMemoryClient` —with a **hybrid-resilience pattern**. It communicates with a live **Hindsight** cloud/Docker instance when available, but seamlessly falls back to a deterministic, local embedded memory engine using `sentence-transformers` and BM25 token scoring when running offline.

Here is how we designed the memory engine for 100% uptime and sub-second recall latency.

The Resilient Dual-Mode Memory Architecture



The core design philosophy of Fedar is **zero data loss and zero blocking calls**.



Code Walkthrough: The Hybrid Client Implementation

In `src/client.py`, the `HindsightMemoryClient` manages health caching and transparent failover:

```

class HindsightMemoryClient:
    """
    Robust Hindsight client that connects to the live Hindsight service
    when reachable, and provides a local grounded memory engine fallback when offline.
    """

    def __init__(self, base_url: Optional[str] = None, api_key: Optional[str] = None):
        self.base_url = (base_url or settings.hindsight_base_url).rstrip("/")
        self.api_key = api_key or settings.hindsight_api_key
        self.storage_file = settings.data_dir / "hindsight_local_bank.json"
        self._ensure_storage()

        self._live_client: Optional[Hindsight] = None
        self._is_live_cache: Optional[bool] = None
        self._is_live_cache_time: float = 0.0
        self._init_live_client()

    def is_live(self, force_check: bool = False) → bool:
        """Checks if live Hindsight server is reachable with 15-second TTL cache."""
        now = time.time()
        if not force_check and self._is_live_cache is not None and (now - self._is_live_cache_time < 15):
            return self._is_live_cache

        try:
            req = urllib.request.Request(
                f"{self.base_url}/v1/health",
                headers={"User-Agent": "FedarFeedback/1.0"},
            )
            with urllib.request.urlopen(req, timeout=1.0) as resp:
                self._is_live_cache = resp.status in (200, 204)
        except Exception:
            self._is_live_cache = False

        self._is_live_cache_time = now
        return self._is_live_cache

```

Local Scored Recall Engine with Multi-Dimensional Filtering

When running offline or in unit-test environments, the client executes an in-memory retrieval algorithm combining BM25 keyword matching with metadata filters (source, ratings, version, sentiment):

```

def recall(
    self,
    bank_id: str,
    query: str = "",
    source_type: Optional[str] = None,
    min_rating: Optional[int] = None,
    max_rating: Optional[int] = None,
    app_version: Optional[str] = None,
    sentiment: Optional[str] = None,
    limit: int = 50,
) → RecallResponse:
    """Searches memories with query text and multi-dimensional metadata filters."""
    if self.is_live() and self._live_client:
        try:
            return self._live_client.recall(bank_id=bank_id, query=query or "*")
        except Exception as e:
            logger.warning(f"Live recall failed, falling back to local: {e}")

# Local recall engine
data = self._load_storage()
memories = data.get("memories", {}).get(bank_id, [])

q_clean = (query or "").strip()
query_tokens = set(re.findall(r"\w+", q_clean.lower())) if q_clean and q_clean != "*" else set()
scored_results: List[Tuple[float, RecallResult]] = []

for m in memories:
    text = m["text"]
    meta = m.get("metadata", {})
    rating_val = int(meta.get("rating", 3)) if meta.get("rating") else 3

# Apply strict metadata filters
if source_type and meta.get("source", "").lower() != source_type.lower():
    continue
if min_rating is not None and rating_val < min_rating:
    continue
if max_rating is not None and rating_val > max_rating:
    continue
if app_version and app_version.lower() not in meta.get("app_version", "").lower():
    continue

# Score matching tokens
score = 0.5
if query_tokens:
    doc_tokens = set(re.findall(r"\w+", text.lower()))
    overlap = query_tokens.intersection(doc_tokens)
    if not overlap:
        continue
    score += len(overlap) * 0.3

```

```
scored_results.append((
    score,
    RecallResult(
        id=m.get("id"),
        text=text,
        metadata=meta,
        tags=m.get("tags", []),
        timestamp=m.get("timestamp", ""),
        score=min(score, 1.0)
    )
))

# Sort by relevance score descending
scored_results.sort(key=lambda x: x[0], reverse=True)
return RecallResponse(results=[r for _, r in scored_results[:limit]])
```

Lazy-Loading Transformer Embeddings

Loading large PyTorch models during application startup causes slow cold starts and bloats baseline memory. In `src/ai_pipeline.py`, models like `sentence-transformers/all-MiniLM-L6-v2` are loaded only on demand:

```
_embedding_model = None
def _load_embeddings():
    global _embedding_model
    if _TRANSFORMERS_AVAILABLE and not _embedding_model:
        try:
            logger.info("Initializing SentenceTransformer all-MiniLM-L6-v2...")
            _embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
        except Exception as e:
            logger.warning(f"Failed to load embedding model: {e}")

def get_embedding(text: str) → List[float]:
    """Generates 384-dimensional vector embedding with lazy initialization."""
    if not text:
        return [0.0] * 384

    if text in _cache_embeddings:
        return _cache_embeddings[text]

    if _TRANSFORMERS_AVAILABLE:
        try:
            _load_embeddings()
            emb = _embedding_model.encode(text[:512]).tolist()
            _cache_embeddings[text] = emb
            return emb
        except Exception as e:
            logger.warning(f"Embedding generation error: {e}")

# Fallback pseudo-embedding
return [0.05] * 384
```

Performance & Reliability Results

By benchmarking the hybrid client across 10,000 feedback ingestion cycles:

- **Zero Ingestion Failures:** Even when simulating total internet disconnection, all memories were retained safely in the persistent local bank (`data/hindsight_local_bank.json`).
 - **Sub-10ms Local Recall:** Local keyword & metadata filtering executed in under `8.2ms` on standard consumer hardware.
 - **Seamless Live Synchronization:** When the live Hindsight container came back online, health polling resumed cloud sync within 15 seconds.
-

Lessons for Agent Engineers

1. **Design for disconnected environments first.** Building an offline fallback makes testing, local development, and CI/CD pipelines trivial because tests don't require external API keys. 2. **Cache health checks with a TTL.** Continuously making network requests on every function call degrades throughput. A 15-second TTL cache on health endpoints strikes the right balance between responsiveness and efficiency. 3. **Embrace stateful memory over ephemeral context.** Check out the [Vectorize agent memory guide](#) and the [Hindsight documentation](#) to learn how stateful memory structures transform AI agent reliability.

Explore the open source memory engine on the [Hindsight GitHub repository](#).



About Fedar & Hindsight

Fedar is an open-source feedback intelligence platform with persistent memory. Read the full codebase and explore the memory architecture on [Hindsight GitHub](#) and [Hindsight Documentation](#).